DrNutrident — Master Prompt for Manus AI

*Paste everything below the line into Manus. Attach: the **sanitized** `backend.zip` (no `.env`), `DrNutrident-Frontend-Specification.pdf`, `DrNutrident-Backend-API-Documentation.pdf`, and the specification document "DrNutrident — Backend Audit & React.js Website Specification" (export it as Markdown or PDF). Before attaching: delete `backend/.env` from the ZIP and rotate every credential that was in it.*

TASK: Build the complete DrNutrident React.js website on the existing backend

1. Your role and mission

You are a senior full-stack engineer. Build a production-oriented **React.js web application** for **DrNutrident, a dental-clinic management platform**, connected to the **existing Node.js/Express/MongoDB backend** in the attached ZIP. Four roles (patient, dentist, receptionist, admin) use **one app** with role-based navigation. Deliver working software, not a mock-up: every visible control must call the real API, and nothing may report success unless the backend or provider confirms it.

Domain warning. Despite the project name, every file describes a **dental** system (Dentist, DentalChart with FDI tooth numbers, TreatmentPlan, Treatment catalog). Do **not** build nutritionist, diet-plan, or nutrition-assessment features: they do not exist in the backend or any document and are out of scope.

2. Inputs and reading order (do this before writing code)

1. Read the attached specification document fully. It contains the audit, the endpoint catalogue (103 endpoints), the finding IDs (S-xx, B-xx, F-xx, C-xx), proposed endpoints (E1–E12) and tasks (T01–T42) referenced below.
2. Read both PDFs. A third PDF, `DrNutrident_Website_Build_Specification.pdf`, sits at the root of the ZIP; read it too.
3. Extract the ZIP and inspect `backend/src` (routes → controllers → models → services). **The code is the source of truth** when it disagrees with any PDF. Do not trust a document claim without finding it in code.
4. Write a short "Inspection report" (what you verified, what differs from the spec) as `docs/INSPECTION.md` before building.

The ZIP has **no tests, no README, no frontend source**. The API Reference mentions a "patient app" and "staff panel"; they were not supplied, so do not look for them.

3. Hard rules

- Never print, log, commit, or hard-code secrets. Never place a secret in a `VITE_*` variable. Create `backend/.env.example` and `web/.env.example` with placeholders only. If a real `.env` is present, do not read values into outputs; tell the user to rotate them.
- No hard-coded sample data, no fake success states, no simulated payments or video calls. If a provider is not configured the API returns 503: show the server's message.
- Do not invent endpoints. If something is missing, either implement it as one of the **proposed** endpoints below (backend patch, with tests) or hide the feature and list it in `docs/KNOWN_LIMITATIONS.md`.
- Do not replace the backend framework, database, auth scheme or response envelope. Make only the **surgical backend patches** in Section 5; keep each patch small, tested and listed in `docs/BACKEND_CHANGES.md` with its finding ID.
- Never trust a client-side payment callback as proof of payment. Never let the client set prices.
- Fix integration errors at the root; do not hide them with mocks.
- If something cannot be tested in your environment (real card payment, two-party video with real devices), say so explicitly in the final audit instead of claiming it works.

4. Project facts you must get right

Backend stack: Node ≥ 22.12, Express 5.2, Mongoose 9 on MongoDB Atlas (DB `dental_clinic`), JWT (HS256) + bcryptjs, express-validator, helmet, express-rate-limit, multer + Cloudinary, Razorpay SDK, Agora token builder, Firebase Admin, Socket.IO 4.8, node-cron, PDFKit, Brevo REST email. Scripts: `npm run dev`, `npm run seed`. Entry: `src/server.js` → `src/app.js`. Express 5 note: `req.query` is read-only (custom sanitizer already handles it).

Base URL: `VITE_API_BASE_URL` (includes `/api`). Two production hostnames appear in the documents: `drnutridentbackend-wj1n.onrender.com` and `drnutrident-backend-wj1n.onrender.com`. Neither is verified. Develop against a **local** backend (`http://localhost:5000/api`) with a dev database; when you can, test `GET /api/health` on both hosts and record which responds. Socket.IO is on the same origin without `/api`.

Envelope: success `{success, message, data?, meta?}`; error `{success:false, message, errors?}` (`errors` only on 422). Always show `message` verbatim; map `errors[].field` to form fields. Pagination exists **only** on `GET /dentists`, `/patients`, `/staff`, `/notifications/me`; everything else returns full arrays: paginate client-side.

Identifier model (critical):

- `User._id` is the **patient id** everywhere: `Appointment.patient`, prescriptions, records, plans, dental chart, invoices, payments, queue patient, and all `/…/patient/:patientId` routes and `GET /patients/:id`. `GET /patients` returns User documents, so its `_id` is already the right id.
- `Dentist._id` (not the user id) is used by `/dentists/:id`, `/dentists/:id/slots`, `/reviews/dentist/:id`, `/queue/dentist/:id`, `Appointment.dentist`, socket room joins. A dentist gets it from `GET /auth/me` → `data.profile._id`.
- `Patient._id` is never used by any API.

Dates: `Appointment.date` is midnight UTC from `YYYY-MM-DD`; format it in UTC. `startTime` / `endTime` are clinic-local `HH:MM` strings. Use `VITE_CLINIC_TIMEZONE` (default `Asia/Kolkata`) to hide slots already past today.

Money: appointment fees are in rupees (`fees.consultationFee + fees.platformFee = totalAmount`; platform fee from `GET /clinic-settings`, default 20; **no tax on appointments**). Razorpay `create-order` returns `amount` in **paise**. Invoices add `ClinicSettings.taxPercentage` server-side. Currency is INR.

Auth: `POST /auth/login` → `{user, accessToken, refreshToken}`; access 7 days, refresh 30 days; `GET /auth/me` → `{user, profile}`; refresh `POST /auth/refresh-token {refreshToken}` returns a new pair; `PATCH /auth/update-password` and `POST /auth/reset-password` also return new tokens that must replace stored ones. No cookies and no server logout. Registration returns tokens immediately; email verification is **not enforced** by the backend, so treat it as a banner/trust badge, never a gate. Rate limits: 20 auth requests per 15 minutes per IP (see patch T03), 300 general.

Roles and ownership: only patients self-register. Dentists/staff are created by admin (`POST /dentists`, `POST /staff`). The first admin comes from `npm run seed` (see T17).

Known response shapes: list endpoints return the array directly in `data`; `GET /dentists/:id` → `{dentist, reviews}`; `POST /dentists` → `{user, dentist}`; `POST /payments/verify` → `{payment, appointment}`; notifications `meta` = `{unreadCount}` only; dental chart `teeth[]` starts **empty** (render all 32 valid FDI teeth as healthy; valid numbers are 11–18, 21–28, 31–38, 41–48).

5. Backend patches to implement first (with Jest + supertest + mongodb-memory-server tests)

The backend is mostly sound, but has confirmed defects that a public website would expose. Implement these before relying on it. Mock Razorpay, Agora, Brevo and Cloudinary in tests.

P0

1. **T02 CORS:** accept `CORS_ORIGINS` (comma list) for Express and Socket.IO, falling back to `CLIENT_URL` / `DENTIST_PANEL_URL`. (Local web runs on port 3000.)
2. **T03 Trust proxy:** `app.set('trust proxy', Number(process.env.TRUST_PROXY)||0)` so rate limiting keys on real client IPs behind Render.
3. **T04 Video join (S-01):** in `joinVideoCall`, allow only the appointment's patient or its dentist's user; require `paymentStatus==='paid'`; allow only from `VIDEO_JOIN_EARLY_MINUTES` (15) before start to `VIDEO_JOIN_LATE_MINUTES` (30) after end; do not stamp `joinedBy*At` for refused callers.
4. **T05 Reschedule (S-02):** restrict to the patient owner (or the dentist owner / admin); return 403 otherwise.
5. **T06 Confirm/complete/no-show (S-03):** dentists may act only on their own appointments; enforce valid source states; confirm requires paid unless explicitly staff-overridden.
6. **T07 Refund (S-04):** admin, or the dentist of that appointment; amount ⩽ remaining; only `paid` payments.
7. **T08 / E1 (F-01, B-04):** add `GET /dentists/admin/all?isVerified&isActive&search&page&limit` (admin) returning all dentists with users and correct `meta.total`; add `escapeRegex` to every user-supplied regex (dentists, patients, staff, clinics).
8. **T09 (S-09):** never return `password` from `POST /dentists` and `POST /staff`.

P1 9. **T10 Payments (S-11–S-13):** idempotent `create-order` (reuse the open order for the same appointment; reject unless appointment is `pending` and payment `pending|failed`); `verify` must check the payment belongs to the caller, fetch the payment from Razorpay and match order and amount, and not confirm a cancelled/rescheduled appointment; webhook: constant-time signature compare on the **raw body**, conditional idempotent updates (only advance `created|pending|failed` → `paid`; never revert `refunded`), handle `refund.processed`, guard a missing secret, and auto-refund a payment captured for an already-cancelled appointment. 10. **T11 Expiry (B-01):** cron job cancelling unpaid `pending` appointments older than `PENDING_APPOINTMENT_TTL_MINUTES` (15), `cancelledBy:'system'` (add to enum), freeing the slot. 11. **T12 Slots (B-02):** exclude past times/dates in the clinic

timezone in `getAvailableSlots`, create and reschedule. 12. **T13 Queue (S-10):** staff or the owning dentist only for `GET /queue/dentist/:id`; socket `queue:join` requires a valid token with an allowed role; unique index `{dentist,date,tokenNumber}` with retry on duplicates; validate check-in input (patientId or walkInName required). 13. **T14 (B-05):** `GET /staff` selects `+active`. 14. **T15 Invoices (B-11):** validators (items non-empty, unitPrice ⩾ 0, patient exists); `GET /invoices` (staff, filters, pagination) and `PATCH /invoices/:id/mark-paid {method}` (add `paidAt`, `paymentMethod`); atomic coupon usage; optional `userId` on `POST /coupons/validate` for staff previews. 15. **T16:** `GET /dentists/me/patients` (E2), `GET /appointments` for receptionist/admin with filters and pagination (E3), `GET /dashboard/receptionist` (E4). 16. **T17 Seed:** `seed.js` must read admin email/password from env and refuse to run without them; no default password.

P2 (do if time allows, else list as limitations): payments list for admin (E5); `couponCode` on `POST /appointments` (E7); profile photo for all roles (E8); care-relationship scoping and ownership checks on plan steps and review replies; audit log; email notifications honoring `notificationPreferences.email`; private file delivery; reminder and `upcoming` date logic; OTP attempt limits; dental-chart tooth validation.

If a patch cannot be deployed to the live Render service by the user yet, the frontend must **feature-detect** (a failed `404/403` on a proposed endpoint) and degrade to the documented workaround rather than break.

6. Frontend: stack and architecture (decisions and reasons)

- **Vite + React 18 + JavaScript**, **Tailwind CSS**, **React Router**, **TanStack Query** (loading/empty/error/retry on every list, slot refetch, polling fallback, poll-until-paid), **Zustand** for auth only, **Axios**, **React Hook Form + Zod**, **Recharts**, **socket.io-client 4**, **`agora-rtc-sdk-ng`** (pin an exact version), **date-fns (+ date-fns-tz)**, Radix UI or Headless UI for accessible primitives. Razorpay `checkout.js` is loaded on demand. No Redux, no TypeScript, no heavy UI kit.
- **API layer:** `src/api/client.js` (Axios, 30 s timeout, “server is waking up” notice after 5 s) plus one thin module per backend domain. Request interceptor adds the bearer token. Response interceptor: on 401 from a non-auth URL, run **one shared refresh promise**, store the new pair, retry once; on failure clear the session and go to `/login?reason=expired`. Normalize errors to `{status, message, fieldErrors}`. 429 → disable the button for 60 s. Never retry 4xx.
- **Token storage:** `localStorage` (the backend has no cookie support), cross-tab logout through the `storage` event, no token logging, strict CSP, no `dangerouslySetInnerHTML`; document the trade-off in the README.

- **Guards:** `RequireRole` redirects anonymous users to `/login` with `returnTo`; wrong-role users go to their own home. Role homes: `/app`, `/dentist`, `/reception`, `/admin`.
- **Shared UI:** Button (loading), FormField, Modal/ConfirmDialog, Toast, two **separate** Badges (appointment status and payment status), DataTable (cards on mobile), Pagination, EmptyState, ErrorState, Skeleton, PdfDownloadButton, FileDropzone, SlotPicker, ToothChart, NotificationBell.
- **Design:** trustworthy clinical look; primary `#0E7C86`, navy headings `#0B2545`, slate body, light borders, 12 px cards, Inter; WCAG 2.1 AA; never colour alone; visible focus; keyboard-operable slot picker, tabs, tooth chart; `prefers-reduced-motion`; mobile bottom tabs for patients `<md`, sidebar `≥lg`; tables become cards on mobile. No official branding exists: do not invent a logo or claims.
- **Structure:** `web/src/{api,store,routes,layouts,pages/{public,auth,patient,dentist,reception,admin,consultation,shared},features/{booking,payment,video,queue,dental-chart,availability},components,hooks,utils}`; tests under `web/tests`.

7. Screens to build (all backed by real endpoints)

Public: `/` (hero with Book CTA, featured dentists from `GET /dentists`, services from `GET /treatments`, contact from `GET /clinic-settings`; hide any section with no data; no invented testimonials), `/about`, `/services`, `/dentists` (filters in the query string: search, specialization, city, minRating, maxFee, sort, page), `/dentists/:id` (profile, reviews, 14-day slot picker), `/pricing` (derived from dentists and settings), `/contact` (details; `mailto:` unless E9 exists), `/privacy` and `/terms` (placeholders; legal text comes from the client), `/login`, `/register`, `/verify-otp`, `/forgot-password`, `/reset-password`.

Patient `/app`: dashboard; `/app/book/:dentistId` (stepper: slot → mode/reason/family member → fees → pay → result); appointments (tabs, status filter) and detail (cancel with reason and refund-policy copy, reschedule → navigate to the **new** appointment id, join video, one review); prescriptions (+ authenticated PDF download); medical records (list, filter by type, upload, delete; Cloudinary URLs open directly); treatment plans; dental chart (read-only); invoices (+PDF); payments history; family members; profile (medical info, photo via multipart `photo`); notifications (poll 30 s, badge from `meta.unreadCount`, `email` preference labelled not yet active); change password.

Dentist `/dentist`: first login with empty `weeklyAvailability` forces `/dentist/availability`. Dashboard (`/dashboard/dentist/me`); schedule with date/status filters and actions (confirm, complete, no-show, cancel, join video); availability editor (weekly windows, slot duration, leave dates, accepting toggle; validate `HH:MM`,

start<end, no overlaps); profile edit; patients (interim: unique patients from `GET /appointments/dentist/me`; use E2 when available); patient clinical view with tabs: prescriptions (dynamic medicine rows, all four fields required), records (upload), treatment plans (steps from `GET /treatments`, step status updates), dental chart (tap tooth → condition + notes); reviews with reply; queue.

Receptionist `/reception`: dashboard (E4 or today's queue per dentist), patients list/detail (read-only clinical tabs, check-in shortcut), live queue (check in a registered patient or walk-in, optional appointment id; call, skip, complete), invoice creation (line items, coupon preview, show **server-calculated** totals after create, PDF), appointment support (lookup by id; list/search when E3 exists; confirm, cancel, no-show).

Admin `/admin`: dashboard KPIs and three charts (zero-fill the weekly series and drop future dates client-side); dentist management (list via E1, create, verify, optional clinic assignment through `PATCH /clinics/:id/assign-dentist`); staff management (create, list, filter, activate/deactivate); patients; treatment catalog CRUD; coupons CRUD; clinics CRUD (optional section); clinic settings; payments + refunds only if E5 exists. Confirm before every delete. Do not build clinic-wide review aggregation, a clinic-breakdown chart or a `clinicId` dashboard filter (not in the backend).

8. Razorpay (real, test mode)

Flow, exactly:

1. `POST /appointments` → pending appointment (use the server's `fees` from then on).
2. `POST /payments/create-order {appointmentId}` → `{orderId, amount(paise), currency, keyId, paymentId}`.
3. Load `https://checkout.razorpay.com/v1/checkout.js`; open Checkout with `key=keyId`, `order_id=orderId`, `amount`, `currency`, clinic name, user prefill, `handler`, `modal.ondismiss`, and a `payment.failed` listener.
4. On success call `POST /payments/verify` with `razorpay_order_id`, `razorpay_payment_id`, `razorpay_signature`.
5. On dismiss, failure, network error or timeout: **never show success**. Poll `GET /appointments/:id` every 3 s for up to 30 s; show confirmed only if `paymentStatus==='paid'`; otherwise show "payment not completed" with a retry that calls create-order again.
6. State machine: `idle → creating-appointment → creating-order → checkout-open → verifying → confirmed | confirming-in-background | failed | cancelled`.
7. Handle 400/409 at booking ("slot was just taken": refetch slots), 503 (payments unavailable).

The webhook (`POST /api/payments/webhook` , raw body, `RAZORPAY_WEBHOOK_SECRET` , events `payment.captured` , `payment.failed` , plus `refund.processed` after T10) is server-to-server and **never called from the frontend**. The keys in the ZIP are test-mode (`rzp_test_…`): use Razorpay test cards/UPI only; never run real transactions in automated tests. Cancellation inside `ClinicSettings.cancellationFullRefundHours` (default 24 h) triggers an automatic full refund; later cancellations stay `paid` for manual handling: write UI copy accordingly. Coupons do not apply to appointments until E7 exists; hide the booking coupon field until then.

9. Video consultations (real, Agora)

`POST /appointments/:id/video/join` → `{appId, channelName: 'appt_<id>', uid, token, expiresInSeconds: 7200}` (503 if Agora is not configured). Implement `/consultation/:id/lobby` and `/consultation/:id` for patient and dentist:

- Show “Join video call” only for `mode==='video' && status==='confirmed'` , enabled inside the allowed window (countdown otherwise).
- Lobby: `AgoraRTC.checkSystemRequirements()` , camera/microphone permission and test; on denial explain how to allow and offer audio-only.
- Call: `AgoraRTC.createClient({mode:'rtc', codec:'vp8'})` , `client.join(appId, channelName, token, uid)` , publish local tracks, subscribe on `user-published` , show “Waiting for the other participant” until a remote user appears, mute/unmute, camera on/off, leave, network-quality indicator.
- Failures: `connection-state-change` → “Reconnecting…”; `token-privilege-will-expire` → call join again and `client.renewToken` ; `user-left` notice; hard failure → error panel with retry; on leave unpublish, close tracks, `client.leave()` .
- After the call the **dentist** marks the appointment completed (existing endpoint). No recording, no chat.
- Camera/microphone need HTTPS (localhost exempt). Verify with Chromium fake-media flags that two contexts see each other, and that a third user receives 403. Be explicit about what you could not test with real devices.

10. Live queue

Connect `io(apiOrigin, {auth:{token: accessToken}})` , emit `queue:join` with the **Dentist id**, listen for `queue:updated` (`{action, entry}` ; `entry` is not populated, so invalidate and refetch the list), emit `queue:leave` on unmount. If the socket is disconnected, poll the list every 8 s and show a small “live updates paused” indicator. Dentists see only their own queue; receptionists pick a dentist from `GET /dentists` .

11. Environment configuration

Backend (placeholders only in `.env.example`): `PORT, NODE_ENV, CORS_ORIGINS, CLIENT_URL, DENTIST_PANEL_URL, TRUST_PROXY, MONGO_URI, JWT_SECRET, JWT_EXPIRES_IN, JWT_REFRESH_SECRET, JWT_REFRESH_EXPIRES_IN, BCRYPT_SALT_ROUNDS, RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET, CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET, BREVO_API_KEY, EMAIL_FROM_ADDRESS, EMAIL_FROM_NAME, FIREBASE_SERVICE_ACCOUNT_BASE64, AGORA_APP_ID, AGORA_APP_CERTIFICATE, PENDING_APPOINTMENT_TTL_MINUTES, VIDEO_JOIN_EARLY_MINUTES, VIDEO_JOIN_LATE_MINUTES, ADMIN_EMAIL, ADMIN_PASSWORD (seed only)`. JWT secrets must be ≥ 64 random bytes. Frontend (public only): `VITE_API_BASE_URL`, `VITE_SOCKET_URL` (optional), `VITE_CLINIC_TIMEZONE`. Razorpay key id and Agora app id arrive from the API; never put secrets in the frontend.

12. Testing you must run and report

- **Backend (Jest/supertest/memory-server):** auth errors; authorization matrix (cross-patient reschedule, cross-dentist confirm/complete, non-participant/admin/receptionist video join, cross-dentist refund, patient reading a queue); concurrent booking (one 201, one 409); past-slot rejection; pending expiry; payments (idempotent order, bad/forged signature, other patient's order, duplicate and post-refund webhooks); video window, unpaid and 503; upload type/size; no password in create responses; regex-injection search.
- **Frontend (Vitest + RTL + MSW):** forms and 422 mapping; guards; two parallel 401s → one refresh; slot past-time filtering; checkout state machine (success, dismiss, failure, verify timeout); ToothChart renders exactly the 32 FDI teeth; blob download error handling; loading/empty/error on every list; layouts at 375/768/1280.
- **E2E (Playwright, local stack, Razorpay test mode, real Agora project):** register→OTP→login; admin creates and verifies a dentist who then appears in search; book→pay (test)→confirmed; closed checkout stays pending; slot conflict; complete→review once; early vs late cancellation refund behaviour; prescription+PDF, record upload, plan, tooth update; check-in→call→complete live in two browsers; video with a refused third user; cross-role route and API access; expired-token refresh.
- Run `npm run build` for the web app and all suites; fix failures instead of weakening tests.

13. Required documentation and deliverables

`README.md` (install, run, build, env, architecture, token-storage trade-off), `docs/INSPECTION.md`, `docs/BACKEND_CHANGES.md` (each patch with finding ID, files, tests), `docs/API.md` (endpoints actually used), `docs/SETUP.md`, `docs/DEPLOYMENT.md` (static host with SPA fallback and HTTPS, CSP allowing the API, `checkout.razorpay.com`, Agora and Cloudinary; production `CORS_ORIGINS`; `TRUST_PROXY=1`; Razorpay live-mode activation with a new live webhook; Agora production settings; additive-only migrations and the Queue unique-index precheck; monitoring; rollback), `docs/KNOWN_LIMITATIONS.md`, and both `.env.example` files. Do not assume a frontend host; none is specified.

14. Uncertainties and the safe approach for each

- **Which Render host is live:** use env; test health on both; record the result.
- **Brevo sender verification / OTP delivery:** unknown until run; the UI must work when no email arrives (resend with cooldown; verification is not a gate).
- **Render cold starts:** handled by the timeout and “waking up” notice.
- **Server timezone:** assumed UTC; use date-only strings for API dates.
- **Whether patched backend is deployed:** feature-detect proposed endpoints and use the documented workaround.
- **Phone verification (Firebase):** hide in v1.
- **Privacy/health-data compliance:** do not claim compliance; list recommended legal review in the limitations.

15. Out of scope

Nutrition/diet features, in-app chat, web push, SMS/WhatsApp, recording, native apps, clinic-breakdown analytics, and anything not listed above unless the user confirms it.

16. Final report (required)

End with an audit containing: (a) every feature marked **Done and tested / Done, not testable here / Partial / Not done**, with the reason; (b) every backend patch applied, with finding IDs and test names; (c) test results (counts, failures, how you ran video and payments); (d) credentials and accounts the user must supply or rotate (Razorpay live keys and webhook, Agora, Cloudinary, Brevo sender, Mongo, JWT secrets, first admin); (e) remaining risks from the specification’s findings register that you did not fix; (f) exact steps to run, build, test and deploy. Do not describe unverified behaviour as working.